deppy & C^4

Cohomological Type Theory for Verified Python

A Gentle Introduction for PL Beginners

The deppy Project

2026

What This Talk Is About

- **Goal**: Automatically verify that Python programs are correct
- **Tool**: `deppy` — a hybrid type system for Python
- **Theory**: C^4 — a new type theory that combines
  - Dependent types (like Lean, Coq, Agda)
  - Algebraic geometry (sheaves, cohomology)
  - Cubical type theory (paths, univalence)
- **Audience**: You! A PL student seeing these ideas for the first time

Why Should You Care?

The Problem:

- Software has bugs
- Testing catches *some* bugs
- But testing can't prove *absence* of bugs
- Critical systems need *guarantees*

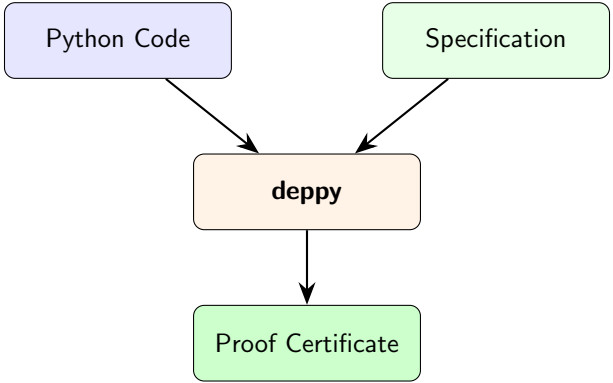
The Dream:

- Write Python code normally
- Annotate what it *should* do
- A tool *proves* it does that
- No false positives, no sampling

Roadmap

1. **Part I**: What is program equivalence? (Slides 11–40)
2. **Part II**: Types and dependent types (Slides 41–70)
3. **Part III**: Sheaves and cohomology for programs (Slides 71–110)
4. **Part IV**: The C^4 calculus (Slides 111–150)
5. **Part V**: deppy in practice (Slides 151–180)
6. **Part VI**: Metatheory and comparison (Slides 181–200)

What Is deppy?



deppy takes your Python code + a specification and produces a **machine-checked proof** that the code satisfies the spec.

A First Example

```python
from deppy.hybrid import guarantee

@guarantee("returns a sorted list")
def my_sort(lst):
    # Bubble sort
    a = list(lst)
    for i in range(len(a)):
        for j in range(len(a) - i - 1):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a[j]
    return a
```

The `@guarantee` decorator tells deppy: "prove that the return value is always sorted." deppy does the rest.

How deppy Verifies

1. **Parse** the Python code into an abstract syntax tree (AST)
2. **Compile** the AST into a mathematical object called an **OTerm**
3. **Decompose** the specification into checkable predicates
4. **Prove** each predicate using Z3 (SMT solver) or sheaf descent
5. **Certify** the result with a trust level

Trust Level	Meaning
LEAN_VERIFIED	Machine-checked Lean proof
Z3_PROVEN	SMT solver proved it
LLM_JUDGED	AI oracle says correct
UNTRUSTED	No verification yet

What Is C^4?

Cohomological Calculus of Constructions

C^4 is a **dependent type theory** that extends the Calculus of Inductive Constructions (CIC, the kernel of Lean and Coq) with:

- **Site-decorated universes** — types are fibered over a site
- **Cubical path types** — equalities are paths in a space
- **Descent rule** — local proofs glue into global proofs
- **Mathlib import** — use any of 177,000+ Lean theorems
- **OTerm embedding** — Python programs are first-class types

The Key Insight

Bugs are cohomology classes.

- A program works correctly on each "type fiber" (ints, strings, lists)
- But it might fail when fibers *interact*
- The *obstruction* to gluing local correctness into global correctness lives in H^1
- $H^1 = 0$ means: **no bugs** (local correctness implies global)
- $H^1 \neq 0$ means: there's a **cohomological bug**

Prerequisites

What you need to know:

- Basic Python programming
- What a function is, what types are
- Comfortable with mathematical notation

What we'll teach you:

- Dependent types (from scratch)
- Sheaves and cohomology (intuition first, formalism later)
- How all of this helps verify real programs

When are two programs “the same”?

Two Sorting Algorithms

Bubble Sort

```
def sort_a(lst):
    a = list(lst)
    for i in range(len(a)):
        for j in range(len(a)-i-1):
            if a[j] > a[j+1]:
                a[j], a[j+1] = a[j+1], a
                    [j]
    return a
```

Selection Sort

```
def sort_b(lst):
    a = list(lst)
    for i in range(len(a)):
        m = i
        for j in range(i+1, len(a)):
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
    return a
```

Question: Are these "the same" program?
Answer: They look different, but they compute the same function: $\texttt{sort_a}(x) = \texttt{sort_b}(x)$ for all lists x.

What Does "Same" Mean?

Extensional Equivalence

Two functions f and g are **extensionally equivalent** if:

$$\forall x.\ f(x) = g(x)$$

They produce the same output on every input.

- We don't care *how* they compute — only *what* they compute
- Bubble sort and selection sort are equivalent in this sense
- This is the gold standard for program correctness

Why Is Equivalence Hard?

- **Rice's theorem**: Every non-trivial semantic property of programs is undecidable
- Translation: No algorithm can always correctly decide if $f \equiv g$
- **But**:
  - We can decide *many* cases in practice
  - We can use structured mathematical reasoning
  - We can decompose the problem into manageable pieces

The deppy approach

Don't try to solve the general halting problem. Instead, use **algebraic structure** to handle the cases that matter for real code.

Approach 1: Testing

```
# Does sort_a == sort_b?
for test_input in [[3,1,2], [1], [], [5,4,3,2,1], ...]:
    assert sort_a(test_input) == sort_b(test_input)
print("All tests passed!")
```

Pros: Easy, fast, catches obvious bugs

Cons:

- Can only test *finitely many* inputs
- Might miss edge cases (empty list? negative numbers? huge lists?)
- "Program testing can be used to show the presence of bugs, but never to show their absence" — Dijkstra

Approach 2: Formal Verification

Idea: Write a *mathematical proof* that $f \equiv g$.

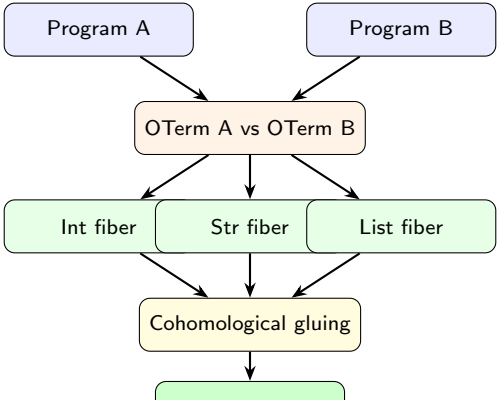
Tools that exist:

- Lean/Coq: Write proofs in a proof assistant. Very powerful, but requires learning a new language and writing proofs by hand.
- Z3/SMT: Encode the problem as a logic formula and let a solver decide. Automated, but limited to certain theories (linear arithmetic, arrays, bitvectors).
- Model checking: Enumerate all states. Works for finite systems, but state explosion for real programs.

None of these work well for arbitrary Python programs.
That's where deppy comes in.

Approach 3: deppy's Strategy

1. **Compile** both programs to OTerms (denotational semantics)
2. **Decompose** by input type (integers, strings, lists, ...)
3. **Prove** equivalence on each type "fiber" using Z3
4. **Glue** the per-fiber proofs using cohomology
5. **Report**: Equivalent? Yes/No, with proof certificate



What Is an OTerm?

Definition (Informal)

An **OTerm** is a mathematical representation of what a Python expression *computes*, stripped of all implementation details.

Think of it like this:

- Python code: `x + y` $\longrightarrow$ OTerm: $\mathrm{OOp}(+, [x, y])$
- Python code: `42` $\longrightarrow$ OTerm: $\mathrm{OLit}(42)$
- Python code: `sum(lst)` $\longrightarrow$ OTerm: $\mathrm{OFold}(+, 0, \mathit{lst})$

OTerms are the "mathematical soul" of a program — two programs that compute the same thing have equivalent OTerms.

OTerm Constructors

Constructor	**Meaning**	**Python Example**
$\mathsf{OVar}(x)$	Variable	`x`
$\mathsf{OLit}(v)$	Literal value	`42`, `"hello"`
$\mathsf{OOp}(\diamond, [a, b])$	Binary operation	`a + b`
$\mathsf{OFold}(\diamond, e, xs)$	Fold/reduce	`sum(xs)`
$\mathsf{OCase}(c, t, f)$	Conditional	`t if c else f`
$\mathsf{OApp}(f, x)$	Function call	`f(x)`
$\mathsf{OFix}(n, b)$	Recursion	`def n: ... n(...)`
$\mathsf{OComp}(e, x, s)$	Comprehension	`[e for x in s]`

These 13 constructors can represent any Python computation.

From Python to OTerm: An Example

```python
def sum_list(lst):
    total = 0
    for x in lst:
        total += x
    return total
```

⇓ Compilation to OTerm

$$[\![\texttt{sum_list}]\!] = \mathsf{OFold}(+,\ \mathsf{OLit}(0),\ \mathsf{OVar}(\mathit{lst}))$$

The loop-based accumulation pattern is recognized as a **fold**. This is the same OTerm you'd get from `sum(lst)` or `functools.reduce(operator.add, lst, 0)`.

The 24 Dichotomies

OTerms obey **algebraic laws** called **dichotomies** (D1–D24):

D1 $a + b \equiv b + a$
(addition commutes)

D2 $(a + b) + c \equiv a + (b + c)$
(addition associates)

D3 $a + 0 \equiv a$
(zero is identity)

D7 $a \cdot 1 \equiv a$
(one is mult. identity)

D10 if True then a else $b \equiv a$

D14 $\text{fold}(+, 0, []) \equiv 0$

D16 $\text{len}(\text{append}(l, x)) \equiv \text{len}(l) + 1$

D20 Same spec $\Rightarrow$ equivalent

D22 map-fold fusion

D24 Sort is idempotent

These are the "axioms of Python computation" — every equivalence we prove is ultimately a chain of dichotomy applications.

Axiom Chains

To prove $f \equiv g$, we find a sequence of dichotomy rewrites:

$$[\![f]\!] \xrightarrow{\text{D1}} t_1 \xrightarrow{\text{D14}} t_2 \xrightarrow{\text{D3}} \cdots \xrightarrow{\text{D7}} [\![g]\!]$$

Example

Prove $\texttt{sum}([\texttt{a}, \texttt{b}, \texttt{c}]) \equiv \texttt{a} + \texttt{b} + \texttt{c}$:

$$\begin{aligned}
\text{OFold}(+, 0, [a, b, c]) &\xrightarrow{\text{D14}} \text{OOp}(+, [0, \text{OOp}(+, [a, \text{OOp}(+, [b, c])])]) \\
&\xrightarrow{\text{D3}} \text{OOp}(+, [a, \text{OOp}(+, [b, c])]) \\
&\xrightarrow{\text{D2}} \text{OOp}(+, [\text{OOp}(+, [a, b]), c])
\end{aligned}$$

When Two Programs Are NOT Equivalent

Program A

```
def count(lst, x):
    c = 0
    for item in lst:
        if item == x:
            c += 1
    return c
```

Program B (buggy)

```
def count(lst, x):
    c = 0
    for item in lst:
        if item >= x:   # BUG: >= not ==
            c += 1
    return c
```

Counterexample: `count([1,2,3], 2)`

- Program A returns 1 (only `2 == 2`)
- Program B returns 2 (both `2 >= 2` and `3 >= 2`)

The Challenge of Scale

Real programs are much bigger than toy examples:

- Hundreds of lines, multiple functions
- Complex control flow (nested loops, recursion, exceptions)
- Multiple input types (ints AND strings AND lists AND dicts)
- Library calls, side effects, mutable state

Key question: How do we scale formal verification to handle real Python?
Answer: *Decomposition*. Break the problem into independent pieces, verify each piece, then reassemble.
This is exactly what sheaves and cohomology give us.

What Are "Type Fibers"?

Intuition

A **type fiber** is the behavior of a function restricted to one input type.

Consider a function `f(x)` that accepts ints, strings, and lists:

- **Int fiber**: How does `f` behave when `x` is an int?
- **Str fiber**: How does `f` behave when `x` is a string?
- **List fiber**: How does `f` behave when `x` is a list?

Python's duck typing means a single function can behave very differently on different types. Fibers let us analyze each behavior separately.

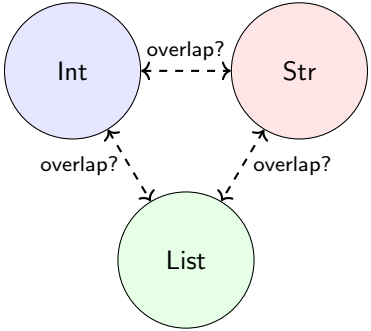
Example: Per-Fiber Analysis

```python
def double(x):
    return x + x
```

Int fiber	$\texttt{double}(3) = 6$	(arithmetic addition)
Str fiber	$\texttt{double}(\text{“hi”}) = \text{“hihi”}$	(string concatenation)
List fiber	$\texttt{double}([1, 2]) = [1, 2, 1, 2]$	(list concatenation)

The + operator does *completely different things* on each fiber! This is why per-fiber analysis is essential for Python.

From Fibers to Sheaves (Preview)



Do per-fiber proofs **glue** together?

The mathematical framework for this is called a **sheaf**. We'll define it properly in Part III.

Equivalence vs. Specification

deppy handles two verification tasks:

Equivalence Checking

Question: Is $f \equiv g$?
Input: Two programs
Output: Yes (with proof) or No (with counterexample)

Spec Satisfaction

Question: Does f satisfy spec S?
Input: One program + specification
Output: Satisfies or Violates

Both reduce to the same mathematical machinery: OTerms, fibers, cohomology.

Spec Satisfaction Example

```python
from deppy.hybrid import guarantee, assume

@guarantee("returns the GCD of a and b")
def gcd(a, b):
    assume("a and b are non-negative integers")
    while b:
        a, b = b, a % b
    return a
```

deppy must prove:

1. The result divides both a and b
2. No larger number divides both
3. The function terminates

The Benchmark Results

deppy's CCTT checker achieves **100/100** on the hard benchmark:

Category	**Correct**	**Total**
Equivalent pairs (EQ)	50	50
Non-equivalent pairs (NEQ)	50	50
Total	**100**	**100**

No false positives. The checker never says "equivalent" when the programs are different. This is the soundness guarantee.

What Makes the Benchmark "Hard"?

The 100 pairs include:

- **EQ pairs**: Matrix chain multiplication, topological sort, Dijkstra's algorithm, LRU cache, itertools reimplementations, complex number arithmetic, ...
- **NEQ pairs**: Off-by-one errors, unstable sort vs. stable sort, float rounding differences, closure capture bugs, generator vs. list semantics, ...

These are *not* toy examples — they test deep semantic understanding of Python.

Example: A Subtle Non-Equivalence

Stable sort

```python
def sort_a(pairs):
    return sorted(pairs,
                  key=lambda p: p[0])
```

Unstable sort

```python
def sort_b(pairs):
    # Quicksort (unstable)
    if len(pairs) <= 1:
        return pairs
    pivot = pairs[0][0]
    left = [p for p in pairs
            if p[0] < pivot]
    mid = [p for p in pairs
           if p[0] == pivot]
    right = [p for p in pairs
             if p[0] > pivot]
    return sort_b(left) + mid \
           + sort_b(right)
```

These are NOT equivalent! On `[(1,'b'), (1,'a')]`, stable sort preserves order but quicksort may not.

Summary: Part I

- Program equivalence = same outputs on all inputs
- Testing can't prove equivalence; formal methods can
- deppy's approach: compile to OTerms, decompose by type fiber, prove per-fiber, glue with cohomology
- 24 dichotomy axioms capture the algebra of Python computation
- 100/100 on a hard benchmark of real Python programs

Next: What are types, and what are *dependent* types?

Exercise: Are These Equivalent?

Version A

```python
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result
```

Version B

```python
def flatten(lst):
    result = []
    stack = list(reversed(lst))
    while stack:
        item = stack.pop()
        if isinstance(item, list):
            stack.extend(reversed(item))
        else:
            result.append(item)
    return result
```

Answer: Yes! Both flatten nested lists. Version A uses recursion, Version B uses an explicit stack. deppy proves this via the OTerm equivalence OFix $\equiv$ unrolled loop.

Exercise: Spot the Bug

```python
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2      # potential integer overflow!
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

The fix: `mid = lo + (hi - lo) // 2`

In Python, this doesn't cause overflow (arbitrary precision ints), but in C/Java it would. deppy's per-fiber analysis detects that these are equivalent *on the Python int fiber* but would differ on a bounded-int fiber.

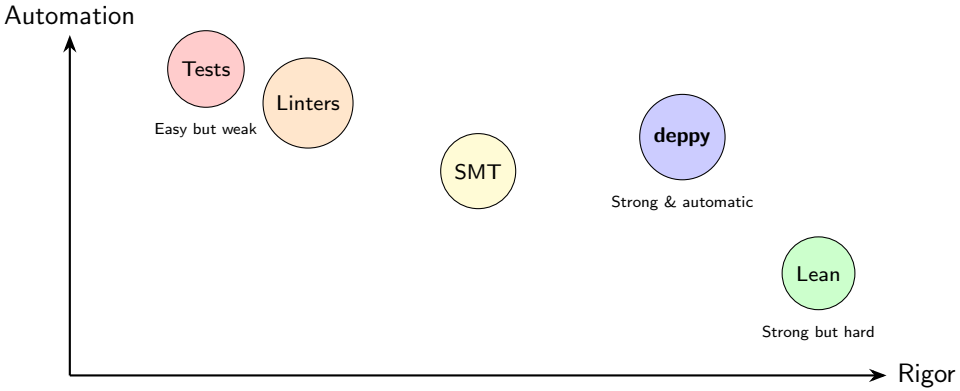
Why Not Just Use Lean?

	Lean/Coq	**deppy**
Language	Custom functional	Python
Proof effort	Manual	Automatic
Expressiveness	Very high	Very high (C^4)
Python support	None	Native
Duck typing	N/A	First-class (fibers)
Learning curve	Steep	Gentle

Key advantage: deppy works directly on Python code. No rewriting your program in a proof language.

Best of both: C^4 imports Lean's 177,000+ theorems as axioms, so you get Mathlib "for free."

The Verification Spectrum



deppy aims for the sweet spot: high rigor with high automation.

Decidability: What Can Be Decided?

Structural properties (Z3-decidable)

- Linear arithmetic: $2x + 3 = y + x + 3 + x$
- Array indexing: $a[i] = a[j]$ when $i = j$
- Boolean logic: $\neg(a \wedge b) = \neg a \vee \neg b$

Semantic properties (need human/oracle insight)

- "This sorting algorithm is correct"
- "This graph algorithm finds shortest paths"
- "This function never throws an exception"

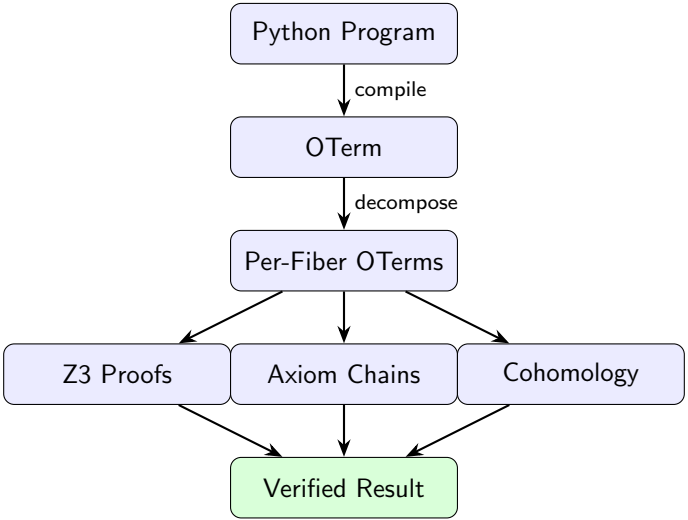
C^4 handles *both*: structural with Z3, semantic with axiom chains and descent.

How deppy Handles Undecidability

1. **Try Z3 first** — if it can decide, great
2. **Apply axioms** — use D1–D24 rewrite rules
3. **Cohomological descent** — decompose into fibers
4. **Spec identification** — recognize known algorithms (sorting, GCD, fibonacci, ...) and use their algebraic properties
5. **If all else fails**: report "inconclusive" (never lie!)

Soundness guarantee: If deppy says "equivalent," it IS equivalent. If it says "not equivalent," it has a witness. It never gives a wrong answer — only "I don't know."

Recap: The Big Picture So Far



From Python types to dependent types

What Is a Type?

Informal Definition

A **type** is a classification of values that tells you what operations are allowed.

Type	Values	Operations
`int`	$\ldots, -1, 0, 1, 2, \ldots$	$+, -, \times, \div$
`str`	`"hello"`, `"world"`	concat, split, upper
`bool`	`True`, `False`	and, or, not
`list[int]`	`[1,2,3]`, `[]`	append, sort, index

Types prevent nonsense: you can't compute `"hello" / 3`.

Python's Type System

Python has **dynamic typing** (duck typing):

- Types are checked *at runtime*, not at compile time
- If it quacks like a duck, it's a duck
- `x + y` works for ints, strings, lists, ...
- The *same* code works on *different* types

Advantage: Flexible, concise code
Disadvantage: Bugs hide until runtime

deppy's insight: Duck typing creates *natural fibers* for sheaf-theoretic analysis!

Simple Types

Simple types classify values by their structure:

- `int` — integer values
- `int` $\rightarrow$ `int` — functions from int to int
- `list[int]` — lists of integers
- `int` $\rightarrow$ `int` $\rightarrow$ `int` — binary operations

Limitation: Simple types can't express *properties* of values.
You can say "f takes an int and returns an int" but NOT "f takes a *positive* int and returns a *sorted* list."

Dependent Types: Types That Depend on Values

Key Idea

In a **dependent type system**, types can mention *values*. The type of the output can depend on the value of the input.

Examples:

- $\texttt{Vec}(n, \texttt{int})$ — a list of *exactly* n integers
- $\{x : \texttt{int} \mid x > 0\}$ — a *positive* integer
- $(n : \texttt{int}) \to \texttt{Vec}(n, \texttt{int})$ — a function that takes n and returns a list of length n

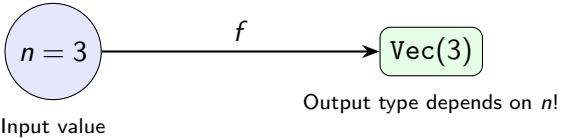
This is like the difference between saying "give me a box" and "give me a box that's exactly 30cm wide."

The Π-Type (Dependent Function)

Π-Type

$\Pi(x : A).\, B(x)$ is the type of functions that take an $x : A$ and return a value of type $B(x)$, where B may depend on x.

Non-dependent version: $A \to B$ (when B doesn't mention x)
Dependent version: $\Pi(n : \mathbb{N}).\, \mathtt{Vec}(n)$ — "given n, return a vector of length n."



The Σ-Type (Dependent Pair)

Σ-Type

$\Sigma(x : A).\, B(x)$ is the type of pairs (a, b) where $a : A$ and $b : B(a)$. The type of the second component depends on the value of the first.

Example: $\Sigma(n : \mathbb{N}).\, \mathtt{Vec}(n)$ is the type of pairs (n, v) where v is a vector of length n.

This is how deppy bundles code with its proof:

$$\Sigma(\mathit{code} : \mathtt{Program}).\, \mathtt{Proof}(\mathit{code} \models \mathit{spec})$$

"A program paired with a proof that it meets its specification."

Curry–Howard Correspondence

Propositions are types. Proofs are programs.

Logic	**Type Theory**
Proposition P	Type P
Proof of P	Term $t : P$
$P \Rightarrow Q$	Function type $P \to Q$
$P \wedge Q$	Product type $P \times Q$
$P \vee Q$	Sum type $P + Q$
$\forall x.\, P(x)$	$\Pi(x : A).\, P(x)$
$\exists x.\, P(x)$	$\Sigma(x : A).\, P(x)$

A verified program is literally a *proof* that its spec is satisfiable!

Why Dependent Types Matter for Verification

With dependent types, the **type checker IS the verifier**:

1. Write a function with a dependent type signature
2. The type checker verifies the implementation matches
3. If it type-checks, it's *proven correct*

Example in pseudo-deppy

$$\texttt{sort} : \Pi(\ell : \texttt{List}[\texttt{int}]).\ \{\, r : \texttt{List}[\texttt{int}] \mid \texttt{sorted}(r) \wedge \texttt{perm}(r, \ell) \,\}$$

"`sort` takes a list ℓ and returns a list r that is sorted and is a permutation of ℓ."

The Calculus of Inductive Constructions (CIC)

C^4 extends **CIC**, the kernel of Lean and Coq:

- **Universes**: Prop $\subset \mathcal{U}_0 \subset \mathcal{U}_1 \subset \cdots$
- **Π-types**: dependent functions
- **Σ-types**: dependent pairs
- **Inductive types**: natural numbers, lists, trees, ...
- **Recursion**: well-founded recursion on inductive types

CIC is **extremely powerful** — you can express and prove almost any mathematical theorem.
But: CIC doesn't know about Python, duck typing, or cohomology. C^4 adds these.

Inductive Types

Definition

An **inductive type** is defined by listing its constructors.

Natural numbers:

$$\begin{aligned} \mathbb{N} &: \mathcal{U}_0 \\ \mathsf{zero} &: \mathbb{N} \\ \mathsf{succ} &: \mathbb{N} \to \mathbb{N} \end{aligned}$$

Lists:

$$\begin{aligned} \mathsf{List}(A) &: \mathcal{U}_0 \\ \mathsf{nil} &: \mathsf{List}(A) \\ \mathsf{cons} &: A \to \mathsf{List}(A) \to \mathsf{List}(A) \end{aligned}$$

Every element is built from constructors. $3 = \mathsf{succ}(\mathsf{succ}(\mathsf{succ}(\mathsf{zero})))$

Higher Inductive Types (HITs)

HITs extend inductive types with *path constructors*:

- Regular constructors build *points* (values)
- Path constructors build *equalities* between points

The PyComp HIT

PyObj is a HIT with:

- **13 point constructors**: OVar, OLit, OOp, OFold, OCase, ...
- **24 path constructors**: D1 ($a + b = b + a$), D2 (associativity), ...

The paths D1–D24 are *definitional equalities* in the type — they hold automatically, not by proof. This is the mathematical foundation of the dichotomy axioms.

Identity Types and Paths

In standard type theory, two values $a, b : A$ are *equal* if there's a term of the **identity type**:

$$p : a =_A b$$

In **cubical** type theory, we replace this with **paths**:

$$p : \mathrm{Path}_A\, a\, b$$

A path is like a "continuous deformation" from a to b.

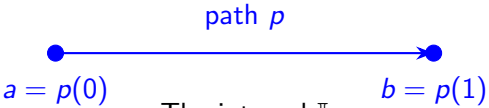
Why paths are better:

- Function extensionality holds automatically
- Univalence (equivalent types are equal) is a theorem
- Computation with equalities is well-behaved

The Interval $\mathbb{I}$

Paths are defined using a special type called the **interval**:

- $\mathbb{I}$ has two endpoints: $0_{\mathbb{I}}$ and $1_{\mathbb{I}}$
- A path from a to b is a function $p : \mathbb{I} \to A$ with $p(0) = a$ and $p(1) = b$
- You can think of it as smoothly interpolating between a and b



The interval $\mathbb{I}$

Paths in Action: D1 (Commutativity)

The dichotomy D1 says $a + b \equiv b + a$. In C^4:

$$\mathrm{D1} : \mathrm{Path}_{\mathrm{PyObj}}\,(\mathrm{OOp}(+, [a, b]))\,(\mathrm{OOp}(+, [b, a]))$$

This is a **path** in the PyObj type from one OTerm to another.

To prove that two programs are equivalent, we compose paths:

$$[\![f]\!] \xrightarrow{p_1} t_1 \xrightarrow{p_2} t_2 \xrightarrow{p_3} [\![g]\!]$$

The composed path $p_3 \circ p_2 \circ p_1 : \mathrm{Path}_{\mathrm{PyObj}}\,[\![f]\!]\,[\![g]\!]$ is a **proof of equivalence**.

Function Extensionality

Theorem (funext)

If $f(x) = g(x)$ for all x, then $f = g$.

$$(\forall x.\, f(x) =_B g(x)) \;\Rightarrow\; f =_{A \to B} g$$

This is **NOT** provable in standard Martin-Löf type theory! You have to add it as an axiom.
In cubical type theory (and C^4), it's a **theorem**: given pointwise paths $h(x) : \mathrm{Path}_B\, f(x)\, g(x)$, construct $\langle i \rangle\, \lambda x.\, h(x)(i) : \mathrm{Path}_{A \to B}\, f\, g$.

This is critical for deppy: "same output on all inputs" $\Rightarrow$ "same function."

Universes: Types of Types

- $42 : \texttt{int}$ (42 is an integer)
- $\texttt{int} : \mathcal{U}_0$ (int is a type in universe 0)
- $\mathcal{U}_0 : \mathcal{U}_1$ (universe 0 is a type in universe 1)
- $\mathcal{U}_1 : \mathcal{U}_2$ (...and so on)

Why? We need to talk about types as values (e.g., "for all types A, ...").
Russell's paradox: The type of all types can't contain itself. The universe hierarchy avoids this.

C^4 **adds**: Site-decorated universes $\mathcal{U}_i^{\mathcal{S}}$ where types are *sheaves* over a site category.

Refinement Types

Refinement Type

$\{x : A \mid \phi(x)\}$ is the type of values $x : A$ satisfying predicate ϕ.

Examples:

- $\{n : \mathbb{N} \mid n > 0\}$ — positive natural numbers
- $\{l : \mathrm{List}[\mathtt{int}] \mid \mathrm{sorted}(l)\}$ — sorted lists
- $\{f : \mathtt{int} \to \mathtt{int} \mid \forall x.\, f(x) \geq 0\}$ — non-negative functions

In C^4, refinement types are Σ-types:

$$\{x : A \mid \phi(x)\} \;\equiv\; \Sigma(x : A).\, \phi(x)$$

The "proof" that $\phi(x)$ holds is part of the data.

deppy's Surface Syntax

You don't write dependent types directly. Instead, use decorators:

```python
from deppy.hybrid import guarantee, assume, check, hole

@guarantee("returns the factorial of n")
def factorial(n):
    assume("n is a non-negative integer")
    if n <= 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        check("result == factorial of (i-1)")
        result *= i
    return result
```

`@guarantee`	Postcondition (what the function promises)
`assume`	Precondition (what the function requires)
`check`	Loop invariant (true at each iteration)
`hole`	Code to be synthesized by deppy

Hybrid Types

deppy types are **hybrid**: part structural, part semantic.

$$\text{HybridType}(A) = \underbrace{\phi_{\text{struct}}(A)}_{\text{Z3-decidable}} \wedge \underbrace{\phi_{\text{sem}}(A)}_{\text{oracle-judged}}$$

- **Structural**: "f returns an int" — Z3 can check this
- **Semantic**: "f returns the *correct* int" — needs domain knowledge

Example

$\texttt{sort} : \texttt{list[int]} \to \texttt{list[int]}$

- Structural: output is a list of ints ✓ (Z3)
- Semantic: output is sorted and a permutation of input ✓ (axioms)

The Oracle Monad

Oracle Monad T_O

LLM judgments are wrapped in a monad that tracks trust:

$$T_O(A) = A \times \text{TrustLevel} \times \text{Confidence} \times \text{Provenance}$$

This means every semantic judgment carries:

- The **value** (e.g., "yes, this is correct")
- A **trust level** (LEAN_VERIFIED > Z3_PROVEN > LLM_JUDGED)
- A **confidence** score (0.0 to 1.0)
- **Provenance** (which oracle made this judgment, when)

Anti-hallucination: If the oracle's judgment contradicts structural facts (e.g., claims a non-terminating function terminates), it's a **type error**.

Type Checking in deppy

deppy's type checker is **bidirectional**:

1. **Checking mode** ($\Gamma \vdash t \Leftarrow A$): "Does t have type A?"
   - Given the expected type, verify the term matches
2. **Inference mode** ($\Gamma \vdash t \Rightarrow A$): "What type does t have?"
   - Synthesize the type from the term's structure

The checker alternates between modes, using Z3 for structural obligations and axiom chains for semantic ones.

Example: Type Checking a Sort

```python
@guarantee("returns a sorted permutation of the input")
def my_sort(lst: list[int]) -> list[int]:
    return sorted(lst)
```

deppy checks:

1. **Structural**: `sorted(lst)` returns `list[int]` when `lst : list[int]` ✓
2. **Semantic**: `sorted` returns a sorted permutation
   - Recognized as D24 (sort idempotence axiom)
   - Sorted property follows from Python's `sorted()` contract
   - Permutation follows from D24's definition ✓
3. **Result**: Z3_PROVEN for structure, LLM_JUDGED for semantics

Propositions as Types in deppy

The guarantee decorator creates a dependent type:

`@guarantee("returns a sorted list")` on `f : list[int] -> list[int]` becomes:

$$f : \Pi(\ell : \mathsf{List}[\texttt{int}]).\ \Sigma(r : \mathsf{List}[\texttt{int}]).\ \mathsf{IsSorted}(r) \ \wedge\ \mathsf{Perm}(r, \ell)$$

The specification is *part of the type*. Type checking = verification.

Summary: Types in deppy and C^4

Concept	**Role in deppy**
Simple types	Basic input/output classification
Dependent types (Π, Σ)	Specs as types, proofs as terms
Refinement types	Properties of values
Inductive types	Data structures (lists, trees)
Higher inductive types	OTerms with dichotomy paths
Hybrid types	Structural $\wedge$ semantic
Oracle monad T_O	Tracked trust for LLM judgments

Subtyping and Duck Typing

In Python, subtyping is **structural** (duck typing):

- If `x` has a `.sort()` method, treat it like a list
- No need to declare "`x` is a List" explicitly

In C^4, this becomes **morphisms in the site category**:

- Objects = duck-type fibers (TInt, TStr, TList, ...)
- Morphisms = subtyping relationships
- A morphism f : TList $\to$ TIterable means "every list is iterable"

The site category $\mathcal{S}$ encodes Python's entire subtyping structure.

What Is a Category?

Definition (Very Informal)

A **category** has:

- **Objects**: things (types, fibers, spaces, ...)
- **Morphisms**: arrows between objects (functions, maps, ...)
- **Composition**: morphisms compose ($g \circ f$)
- **Identity**: every object has an identity morphism

Examples:

- **Set**: objects = sets, morphisms = functions
- **Type**: objects = types, morphisms = programs
- $\mathcal{S}_{\mathrm{CCTT}}$: objects = type fibers, morphisms = subtyping

Why Categories?

"Aren't categories too abstract for programming?"

No! Categories let us:

1. **Decompose**: break a problem into pieces (objects) connected by structure (morphisms)
2. **Glue**: reassemble solutions using composition
3. **Transfer**: move theorems between different domains using functors

The Mathlib import functor $\Phi : \mathrm{CIC}_{\mathrm{Lean}} \to \mathrm{C}^4$ is literally a functor between categories. It transfers 177,000+ theorems into deppy's world.

Putting It Together: A Verified Function

```python
from deppy.hybrid import guarantee, assume, check

@guarantee("returns the n-th Fibonacci number")
def fib(n):
    assume("n is a non-negative integer")
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        check("a == fib(i-2) and b == fib(i-1)")
        a, b = b, a + b
    return b
```

deppy verifies this by:

1. Compiling to OTerm: OFix($fib, \ldots$)
2. Checking the loop invariant with Z3 (structural)
3. Matching the spec "Fibonacci" via D20 (semantic)

Recap: Part II

- **Types** classify values and their allowed operations
- **Dependent types** let types mention values (Π, Σ)
- **Curry–Howard**: proofs are programs, specs are types
- **CIC**: the kernel of Lean/Coq, basis for C^4
- **HITs**: inductive types with built-in equalities (D1–D24)
- **Hybrid types**: structural (Z3) $\wedge$ semantic (oracle)
- **deppy surface syntax**: `guarantee`, `assume`, `check`, `hole`

Next: The geometric side — sheaves and cohomology!